Implementing OpenStreetMap Data for Procedural Game World Generation: A Technical Architecture Report

Introduction to Geospatial World Generation

The paradigm of digital world-building within game engines has undergone a fundamental transformation, transitioning from purely manual, bespoke asset placement to highly automated, procedural generation pipelines driven by real-world geographic data. Historically, modeling an entire city or continent required insurmountable human labor, often resulting in scaled-down, heavily fictionalized approximations of real locations. The integration of Geographic Information Systems (GIS) into game engines has entirely inverted this dynamic. Today, developers leverage planetary-scale datasets to generate physically accurate, 1:1 scale digital twins of real-world environments. At the core of this revolution is OpenStreetMap (OSM), an open-source, community-driven database that maps the physical features of the globe through an intricate topological data structure.[1]

The application of OSM data in video games extends far beyond simple minimaps or 2D overlays. By parsing raw geospatial vectors—including building footprints, road networks, land use delineations, and point-of-interest markers—developers can procedurally extrude three-dimensional geometry, instantiate complex intersection meshes, and apply physically based rendering (PBR) materials dynamically.[2] The utilization of this data has birthed an entire subgenre of location-based and real-world simulation games, seamlessly blending virtual space-time spheres with the context of actual physical environments.[4] This process, however, presents profound mathematical and architectural challenges. Translating abstract geographic topology into renderable, optimized game assets requires sophisticated computational geometry, non-convex polygon tessellation, coordinate system reprojection, and dynamic memory streaming to maintain stable rendering performance across vast virtual spaces.

This report provides an exhaustive, technical examination of the architectures, algorithms, and engine-specific implementations required to translate OSM and allied geospatial datasets into accurate, performant 3D game worlds. The analysis covers the foundational geospatial data models, the algorithmic complexities of procedural mesh triangulation, the mathematical formulations necessary for accurate coordinate projection, and the specific plugin ecosystems surrounding industry-standard game engines such as Unreal Engine, Unity, and Godot. Furthermore, the report examines the evolving landscape of geospatial data formats, contrasting traditional XML and Protocol Buffer implementations with emerging cloud-native formats like GeoParquet managed by the Overture Maps Foundation.

The OpenStreetMap Data Ontology and Architectural Primitives

To effectively translate the physical world into a game engine, the underlying data structure of

OpenStreetMap must be thoroughly understood. OSM does not store pre-computed geometric meshes; rather, it stores a topological representation of the world using three primitive element types: nodes, ways, and relations.[1] This highly flexible, schema-less data model allows for infinite descriptive capacity but demands rigorous pre-processing before it can be utilized in a 3D rendering pipeline. The original raw OSM data requires significant pre-processing to combine fragmented paths of an area, cascade attributes from different layers, and spatially partition data into manageable regional chunks.[3]

Nodes represent the most granular level of geographic information. A node defines a single point in space, represented by geographic coordinates (latitude and longitude) based on the WGS84 ellipsoid.[6] Nodes can exist independently to represent point features—such as a specific tree (natural=tree), a park bench (amenity=bench), or a street lamp (highway=street_lamp)—which are typically converted into instanced static meshes in a game engine.[3] When instancing these objects, advanced parsers like osm2world utilize attachment connectors to calculate local bounds and orientation, ensuring that an instantiated street lamp accurately supports a waste basket attachment based on nested tagging rules.[2]

Ways are ordered lists of nodes. When a way is open (meaning the first and last nodes are distinct), it typically represents a linear feature such as a road, a railway, or a river.[5] In a game engine environment, open ways are generally translated into mathematical splines, along which procedural meshes (like asphalt surfaces or train tracks) are algorithmically extruded.[8] When a way is closed (the first and last nodes are identical), it defines an area or a polygon, such as a building footprint or a park boundary.[3]

Relations are multi-purpose data structures utilized to document complex relationships between two or more data elements, which can include nodes, ways, or other relations.[10] In game world generation, the most critical relation type encountered by parser algorithms is the multipolygon. A multipolygon relation is necessary when the geometry of an area is highly complex—for instance, a building with an interior courtyard, or a vast forest containing a lake.[3]

The processing of multipolygons introduces significant computational overhead. A multipolygon relation consists of a collection of member ways assigned specific roles: outer and inner.[3] The outer ways form the external boundary of the area, while the inner ways form the boundaries of holes or excluded areas within that outer boundary.[3] To form a valid multipolygon that adheres to the Open Geospatial Consortium (OGC) Simple Feature standard, the member ways must combine end-to-end to form one or more closed chains without arbitrary overlapping.[3] Furthermore, exactly two unclosed ways must share a single endpoint; if an endpoint is shared by more than two ways, the topological chain becomes geometrically ambiguous and cannot be closed deterministically.[3] While standard OGC rules strictly prohibit inner rings from touching, OSM data uniquely permits touching inner rings as a mapping convenience, placing the burden on downstream rendering software to process adjacent inner rings as a single combined exclusion zone.[3] Parsing these structures requires robust computational geometry libraries capable of complex boolean operations, ensuring that the spatial area is correctly resolved before mesh generation begins.

Elements in OSM are assigned meaning through tags, which are free-format text fields

consisting of a key and a value separated by an equals sign (e.g., highway=residential or building:levels=5).[1] Because OSM is community-driven, the tagging ontology is entirely unconstrained, containing over 100,000 distinct keys and 190 million distinct tags globally.[11] For game development, this anarchic tagging system necessitates stringent filtering schemas and fallback logic. Procedural generation algorithms rely heavily on specific tags to determine geometric rules. For example, the Simple 3D Buildings specification relies on tags such as building:levels, building:colour, building:material, and roof:shape to determine the extrusion height, texture application, and procedural capping of a structural mesh.[3] Dealing with incomplete or improperly formatted tags is a primary hurdle; procedural engines must employ sophisticated fallback logic, assigning default heights or generic PBR materials when explicit metadata is absent to prevent rendering voids.[13]

To handle the sheer volume of OSM data, developers rely on specialized software libraries and database integrations. While the raw data is formatted in verbose XML (.osm), the industry standard for programmatic access is the Protocolbuffer Binary Format (PBF), which serializes the data for rapid streaming.[3] Integrating OSM into a relational database using PostGIS allows for advanced spatial queries before the data ever reaches the game engine, filtering out unnecessary nodes and optimizing the dataset for real-time applications.[16]

Prominent OSM Parsing Libraries	**Primary Language**	**Functionality & Characteristics**
OsmSharp [15]	C# (.NET)	Streamed architecture with minimal memory footprint, converts native objects to complete instantiations for Unity environments.
libosmium [16]	C++	High-performance toolkit and framework for working with raw OSM data and PBF files.
osm2world [2]	Java (JOGL)	Converts OSM into 3D models (glTF, OBJ) using procedural generation; supports PBR textures and command-line automation.
osmosis [3]	Java	Command-line application for processing, filtering, and

		database integration of raw OSM datasets.
osm4j [16]	Java	Lightweight, cross-platform library for parsing and iterating over OSM PBF archives.

Geodetic Projections and Coordinate Reference Systems

A critical mathematical barrier in geospatial game development is the transformation of spherical, geographic coordinates into the planar, Cartesian space utilized by rendering engines. Game engines operate on a Cartesian X, Y, Z grid, whereas OSM provides data in geographic latitude, longitude, and occasionally altitude.[7]

The Earth is not a perfect sphere; it is an oblate spheroid, most commonly modeled by the World Geodetic System 1984 (WGS84) ellipsoid.[7] The WGS84 model dictates that the Earth is wider at the equator than at the poles, with the semi-major axis $a = 6,378,137.0$ meters and the semi-minor axis $b = 6,356,752.314245$ meters.[7] This geometric reality dictates an inverse flattening ratio of $1/f = 298.257223563$.[7] Consequently, one degree of latitude does not equate to the same linear distance as one degree of longitude, and the linear distance of a longitudinal degree shrinks as one moves from the equator toward the poles.

To render this data in a 3D engine, geographic coordinates must be projected onto a flat plane. A standard approach is the Universal Transverse Mercator (UTM) projection, which divides the Earth into 60 distinct longitudinal zones and applies a localized Cartesian grid to minimize distortion within that specific regional zone.[19] The mathematical conversion from WGS84 latitude (ϕ) and longitude (λ) to UTM Easting (x) and Northing (y) involves complex trigonometric Taylor series expansions to account for the Earth's eccentricity squared (e^2).

The logic executed inside a game engine or its underlying C++ framework to perform localized mercator conversions takes the form of calculations involving the tangent and cosine of the latitude, yielding a local X and Y offset in meters from a defined origin point.[21] For instance, a localized projection algorithm calculates a delta variable (δ) using the arctangent of the sinusoidal variations of the geocentric latitude multiplied by the tangent of the angular basis, ultimately providing a precise floating-point meter value mapped directly to the game engine's coordinate system.[21] In shader programming, developers utilize optimized variants of these trigonometric functions to convert reflection vectors directly into UV coordinates for

latitude-longitude formatted textures, saving instruction overhead in the GPU pipeline.[22]
Once projected into meters, the coordinates must be aligned with the game engine's specific coordinate basis. Unity and Unreal Engine both utilize left-handed Cartesian coordinate systems, but their up-axes differ.[18] In Unity, the $+Y$ axis represents "up," the $+Z$ axis represents "forward" (North), and $+X$ represents "right" (East).[18] Conversely, Unreal Engine defines $+Z$ as "up," $+X$ as "forward" (North), and $+Y$ as "right" (East).[18] Translating geospatial data requires a permutation of the UTM coordinates to match the respective engine's axes, ensuring that building extrusions point toward the sky rather than along the horizon.
Furthermore, traditional game engines historically utilized 32-bit single-precision floating-point numbers to represent positions in world space. A 32-bit float provides approximately 7 decimal digits of precision. Because UTM coordinates are expressed in millions of meters from the equator, directly plugging raw UTM coordinates into a 32-bit engine results in catastrophic spatial jitter, where vertices snap to distinct integer steps, causing meshes to warp, separate, and vibrate erratically during camera movement. To resolve this, modern game architectures employ two primary solutions. The first is World Origin Rebasing, where the developer selects a specific geographic point as the absolute origin $(0, 0, 0)$ of the game level, subtracting this anchor coordinate from all subsequent geospatial imports. The second, more robust solution, adopted natively in Unreal Engine 5 via Large World Coordinates (LWC), is the implementation of 64-bit double-precision floating-point numbers for the core transform architecture, allowing for immense, planet-scale mapping without precision loss or the need for constant origin shifting.[7]

Algorithmic Tessellation of Non-Convex Polygons

Once raw geospatial polygons are parsed and projected into Cartesian space, the engine faces a critical graphics pipeline challenge: rendering the surface areas. Graphics processing units (GPUs) and low-level rendering APIs (such as OpenGL, DirectX, or Vulkan) operate exclusively on triangles. They generally only accept convex polygons natively.[3] A convex polygon is one where all interior angles are less than 180 degrees; drawing a line between any two points inside the shape will never cross the boundary.
However, building footprints and land-use boundaries mapped in OSM are almost exclusively non-convex (concave) and frequently feature inner exclusions (holes).[3] Passing raw OSM coordinate sequences directly to a mesh generator results in broken, intersecting, and visually corrupted geometry.[3] Therefore, the geometry must undergo polygon tessellation, a complex algorithmic subdivision of non-convex shapes into an array of simple convex triangles.[3]
Historical and many contemporary implementations of OSM rendering rely on robust tessellator objects, such as the GLUtessellator found in standard OpenGL bindings (JOGL).[3] The procedural generation workflow requires the developer to feed the contour loops of the OSM multipolygon into the tessellator rather than directly into the rendering pipeline.[3] The

developer defines the primary exterior boundary using a defined contour loop wrapped in gluTessBeginContour and gluTessEndContour, followed by nested contour loops defining the interior holes.[3] The tessellator then calculates the triangulation mathematically, invoking a series of predefined callbacks (such as GLU_TESS_BEGIN, GLU_TESS_VERTEX, and GLU_TESS_END) containing the actual engine-specific vertex creation commands once the subdivision algorithm completes its passes.[3]

A vital component of this tessellation is the application of winding numbers, which mathematically determine what regions of a complex polygon are "solid" (interior) and what regions are "empty" (holes or exterior space).[3] The algorithm achieves this by conceptually casting a ray from an arbitrary point within the plane out to infinity. Starting from zero, the winding number increases by 1 each time the ray crosses a line segment drawn in a counter-clockwise direction, and decreases by 1 when crossing a clockwise segment.[3] By overriding default calculations, developers can pass normal vectors, such as (0, 0, 1), to ensure the front face is always viewed regardless of whether the contour winding is clockwise or counter-clockwise.[3]

Tessellators evaluate these numbers using configurable winding rules, defined via gluTessProperty, which dictate exactly how overlapping paths define solid mass.[3]

Tessellation Winding Rule	Algorithmic Behavior & Application
GLU_TESS_WINDING_ODD	Fills regions where the winding number is odd. This is the default setting and standard method for resolving simple holes within a larger exterior boundary.[3]
GLU_TESS_WINDING_NONZERO	Fills all regions where the winding number is anything other than zero, useful for complex paths that overlap repeatedly.[3]
GLU_TESS_WINDING_POSITIVE	Fills only regions strictly greater than zero.[3]
GLU_TESS_WINDING_NEGATIVE	Fills only regions strictly less than zero.[3]
GLU_TESS_WINDING_ABS_GEQ_TWO	Fills regions where the absolute winding value is greater than or equal to two, effectively capturing only deeply nested overlapping geometries.[3]

Furthermore, dealing with self-intersecting polygons—where an OSM mapper accidentally crossed lines forming shapes like a figure-eight or a star—requires the tessellator to compute intersection nodes dynamically. When a collision between edges is detected, the algorithm

invokes a GLU_TESS_COMBINE callback.[3] This function receives the new X, Y, Z intersection coordinates, pointers to the four existing neighbor vertices, and four weight factors used to mathematically interpolate colors, surface normals, and texture coordinates at the new intersection point, outputting fresh vertex data to prevent the mesh generation from producing non-manifold geometry.[3]

For larger, generalized area fills—such as generating a park terrain mesh bounded by arbitrary curving roads—developers frequently deploy Delaunay Triangulation.[3] This mathematical formulation maximizes the minimum angle of all the angles of the triangles in the triangulation mesh. By enforcing this condition, Delaunay triangulation effectively avoids "skinny" or sliver triangles that cause severe rendering artifacts, lighting anomalies, and poor normal map interpolations in the final game engine representation.[3]

Procedural Generation Methodologies for Urban Infrastructure

With the mathematical foundations of coordinate projection and tessellation established, the procedural generation logic branches into distinct physical features. The two most prominent and computationally intensive features to synthesize are architectural buildings and continuous road networks.

Building Generation and the Straight Skeleton Algorithm

Translating a 2D OSM polygon into a 3D building involves extracting the footprint, tessellating the base polygon into triangles, and extruding the geometry upward along the normal vector. The extrusion magnitude is determined either by parsing the building:levels tag (multiplying the levels by a standard floor height, for example, 3.5 meters) or reading an explicit height tag.[3] To achieve Level of Detail 1 (LOD1), buildings are rendered as simple flat-roofed blocks. Procedural facades can be mapped along the extruded perimeter, utilizing tags like building:material=brick, building:material=concrete, or building:material=glass to assign distinct PBR materials to the wall meshes.[3] Through Python-based data science pipelines utilizing libraries such as OSMnx and PyVista, developers can extract, visualize, and automate the generation of these LOD1 building footprints before importing them into a game engine environment.[26]

While flat-roofed structures are trivial, Level of Detail 2 (LOD2) generation demands complex roof structures, as many OSM entries include tags such as roof:shape=gabled, roof:shape=hipped, or roof:shape=round.[3] Generating pitched roofs for arbitrary, non-convex polygons is a notoriously difficult mathematical problem. The industry-standard solution deployed in procedural generation systems is the Straight Skeleton algorithm.[27]

The Straight Skeleton algorithm operates by shrinking the boundary polygon inward at a uniform rate.[27] As the edges move inward, their connecting vertices trace angular bisectors. During this continuous shrinking process, topological events occur that fundamentally alter the shape of the moving polygon:

1. **Edge Events**: An edge shrinks to a length of zero, collapsing its two adjacent vertices into a single merged point, effectively removing an edge from the structure.

2. **Split Events**: A reflex vertex (an interior angle strictly greater than 180 degrees) moves inward and collides with an opposite edge, splitting the polygon into two independent, smaller polygons that continue to shrink separately until they collapse.[27]

By tracking the Z -axis elevation of these moving vertices based on the rate of shrinkage and the desired trigonometric roof pitch, the algorithm outputs a 3D topological network of intersecting ridges and valleys that perfectly caps the arbitrary footprint.[27]
Applying textures (UV mapping) to these procedurally generated roofs requires precise vector mathematics. Because the triangles generated by the straight skeleton are dynamically formed, standard planar UV projection often causes texture distortion and misalignment across different roof planes. The mathematical solution involves identifying the baseline edge (the eave) of the generated roof section. By taking the cross product of the roof plane's normal vector and a global upward vector, developers derive a horizontal U -axis vector parallel to the eave.[31] The dot product of a vertex's Cartesian position against this calculated U -axis provides the horizontal UV coordinate, while the vertical distance from the eave provides the V coordinate. This ensures that directional shingle or tile textures align perfectly with the slope of the roof geometry regardless of the polygon's spatial orientation.[31]

Road Networks: Spline Topology and Intersection Handling

Roads are parsed from OSM as open ways and translated into spline curves within the game engine.[8] A 2D cross-section template (representing the asphalt, painted lines, curbs, and sidewalks) is algorithmically swept along the spline path to generate a continuous 3D tube or ribbon.[9] However, parsing realistic urban road networks goes beyond simple tube extrusion; academic algorithms utilize the topological networks recorded within OSM to generate sophisticated geometric graphs.[32] Research demonstrates example-based growth algorithms that use patches extracted from source examples (such as Madrid or San Francisco) to generate roads that preserve interesting cultural structures while effectively adapting to the underlying terrain using non-Euclidean metrics.[33]
The paramount challenge in road generation lies at the intersections. OSM data defines intersections merely as shared nodes between two or more ways; it does not provide the geometric dimensions, curb chamfers, or turning radii of the intersection.[6] If spline meshes are blindly extruded toward a shared node, the geometries clip through each other chaotically, creating severe Z-fighting and broken texture alignments.[24] Addressing intersection topology generally follows three architectural approaches:

1. **Node Expansion and Mesh Weaving**: The generation algorithm detects shared nodes and halts the standard spline extrusion a set distance back from the junction point, determined by calculating the combined width of the intersecting roads. A distinct algorithm (such as constrained Delaunay triangulation or an adapted convex hull method) is then utilized to dynamically weave a custom intersection mesh that perfectly bridges the gaps between the halted spline endpoints.[24]

2. **Modular Socket Instantiation**: The system identifies the type of intersection (e.g., T-junction, 4-way cross) by counting the connected ways and measuring their incoming angles. It then spawns a pre-modeled, static intersection mesh and dynamically curves the incoming procedural splines to snap cleanly onto predefined sockets at the edges of the static mesh.[34]
3. **Semantic Lane Modeling**: Highly advanced implementations abandon the monolithic road mesh entirely. Instead, they parse individual lanes based on specific OSM tags (lanes=4, turn:lanes=left|through|right) and project independent, narrower splines for each specific lane, using Bezier curves to directly map permissible traffic paths through the intersection, accommodating bicycle lanes and pedestrian crosswalks automatically.[38]

Topographic Integration and Digital Elevation Models (DEM)

OpenStreetMap is primarily a vector dataset mapping the built environment; it explicitly does not contain dense, ubiquitous elevation data.[41] While certain individual nodes may possess ele tags (usually mountain peaks or significant landmarks), and some topological terrain contour lines exist, relying exclusively on OSM for terrain modeling is entirely insufficient.[41]

To build accurate, non-flat game worlds, developers must fuse the planar OSM vector data with external raster-based Digital Elevation Models (DEMs). Common data sources include the NASA Shuttle Radar Topography Mission (SRTM) dataset, open databases like OpenDEM, or extremely high-resolution local LiDAR scans (such as the AHN viewer datasets detailing the Netherlands with 8 measurements per square meter).[25] Tools such as LASTools are utilized to rapidly process and segment point clouds, deriving precise digital surface models (DSM) and digital terrain models (DTM).[25] These DEM files are typically loaded into the game engine's native landscape or terrain generation system as a heightmap—a dense grayscale image where pixel intensity represents vertical elevation data.[42] Workflows popularized in military simulators like Arma 3 utilize tools such as the Terrain Processor and QGIS to process DEM terrain meshes, add algorithmic noise, and overlay satellite masks before integrating OSM shapefiles representing roads and forests.[19]

Once the underlying terrain mesh is generated and displaced vertically by the DEM, the OSM vectors must be draped or projected onto the new topography to avoid floating structures.[33] This requires an intensive raycasting routine during the procedural generation phase. Before a building footprint is extruded, the system casts downward rays from the polygon's vertices to intersect the terrain mesh, determining the base elevation at multiple points. To prevent buildings from floating above slopes or cutting awkwardly into hillsides, the algorithm must average the intersected heights to establish a flat foundation plane, optionally generating procedural retaining walls or concrete foundations to seamlessly bridge the gap between the building base and the uneven ground below.[46] Road splines similarly adapt by sampling the terrain height at defined intervals, generating continuous elevation profiles that mimic natural highway grading, while cutting through terrain using procedural boolean operations to form

realistic trenches and embankments.

Game Engine Ecosystems and Middleware Implementations

Translating theoretical computational geometry into practical rendering pipelines requires robust middleware and engine-specific tooling. The approaches taken across Unreal Engine, Unity, and Godot vary significantly based on their distinct architectural paradigms, memory management constraints, and rendering philosophies.

The Unreal Engine Ecosystem

Unreal Engine benefits heavily from heavily funded, corporate-backed plugins designed to handle massive geospatial datasets for enterprise and simulation purposes, most notably Cesium for Unreal.[47] Cesium sidesteps the necessity of parsing raw OSM XML locally by streaming data directly from the cloud using the 3D Tiles format.[49] Cesium OSM Buildings provides a global, pre-extruded dataset of building volumes generated from OSM data that is streamed into the engine dynamically based on the camera's frustum and Level of Detail (LOD) requirements.[49]

The plugin provides profound visual integration capabilities, leveraging Unreal's volumetric clouds, fog, and post-processing volumes (such as those demonstrated in Cesium's Mt. Fuji post-processing levels).[50] It further enables developers to access underlying feature metadata encoded in the 3D Tiles, visualizing properties directly through Unreal material layers.[50] Advanced features allow developers to create minimaps via secondary Scene Capture viewports, or hide unwanted details using clipping and Cartographic Polygons to inset high-resolution photogrammetry seamlessly into generic OSM buildings.[50]

However, managing the memory footprint of streaming global OSM data in Unreal Engine is paramount. High-speed camera movements—particularly in flight or combat simulators—can trigger overwhelming tile requests, saturating Video RAM (VRAM) and inducing severe performance spikes.[53] Optimization strategies involve strictly regulating the Maximum Simultaneous Tile Loads (reducing it from default burst values of 50 down to 10) to limit rendering bottlenecks, while proportionally increasing the Maximum Cached Bytes buffer to utilize available system RAM (e.g., allocating 4GB to 6GB of cache).[53]

For developers who require offline generation or deeper programmatic C++ control over the raw OSM vectors in UE, open-source implementations like the Street Map Plugin provide a localized alternative.[54] This plugin ingests raw OSM XML files, stores the relevant topology in specialized FOSMFile structures in system memory, and translates them into native Unreal Engine components, allowing developers to bind logic directly to the geographic data to generate renderable meshes on command.[54]

The Unity Framework

Unity's ecosystem leans heavily toward highly customizable, developer-driven C# procedural generation tools. Because Unity operates on a managed, garbage-collected environment,

parsing massive OSM PBF files directly into memory can trigger severe garbage collection stutters. Middleware solutions mitigate this by providing highly optimized, streamed architectures that sequentially convert raw binary streams into instantiated C# geometry objects with minimal active memory allocation.[15]
Procedural world frameworks encapsulate the heaviest computational geometry tasks within highly performant libraries, exposing the results to Unity via managed bindings.

Unity Procedural OSM Plugins	Primary Functionality & Characteristics
UtyMap [56]	Core logic written in C++11, provides a highly customizable API for procedural world generation at varying zoom levels, integrating OSM and NaturalEarth data.
CityGen3D [46]	Procedurally generates cities based on OSM. Handles complex LOD group instantiations, though requires careful handling of zero-area "junk" OSM entities to prevent blank meshes.
EasyRoads3D [24]	Dedicated road network generator featuring specialized support for junction weaving and intersection mesh alignment.
ActionStreetMap [3]	Framework for building real city environments, utilizing dynamic visitor patterns and Boost Qi parser generators for cross-platform data loading.

These plugins allow the Unity engine to focus purely on instantiating the final meshes. Developers leverage Unity's native Mesh.CombineMeshes API to batch thousands of generated OSM buildings into singular static objects, drastically reducing CPU draw calls and rendering overhead.[24] Modern Unity updates, such as the native Splines package, are increasingly being utilized to wrap OSM road coordinates, combining procedural extrusion algorithms directly with the engine's rendering pipeline to seamlessly generate complex highway interchanges.[8]

The Godot Engine Adaptations

The open-source Godot Engine has seen a surge in community-driven GIS integrations, utilizing GDScript and C++ modules (GDExtension). Tools like the Godot-OSM plugin and the emerging Xoronaut system demonstrate the engine's capability to ingest both vector (OSM) and raster (DEM) GIS files natively into the environment.[60] Community scripts such as

map_viewer.gd allow developers to load raw tilemaps into 2D control nodes for interactive panning and zooming capabilities.[62] Due to Godot's node-based hierarchy, massive city generation often demands chunking systems. Developers utilize background threads to parse geographic data, yielding procedurally generated MeshInstance3D nodes attached dynamically to grid-based Area3D chunk managers to enable infinite world streaming without dropping active frame rates.[63] Workflow recommendations often suggest utilizing the Java OpenStreetMap Editor (JOSM) to repair incomplete local data before parsing it into Godot, ensuring the procedural mesh algorithms receive manifold topological paths.[64]

Next-Generation Architectures: Cloud-Native Formats and Overture Maps

Despite the programmatic efficiency of parsing PBF files in languages like C++ or Java, the format remains a serialized topological stream; it does not natively support advanced spatial indexing or column-based querying. In response to the friction of processing raw OSM data for enterprise and massive simulation environments, the Overture Maps Foundation—a consortium including major technology entities such as Amazon, Meta, Microsoft, and TomTom—was formed to release curated, pre-processed datasets that combine OSM data with supplementary corporate data sources.[65]

Overture utilizes GeoParquet, an open-source extension of the Apache Parquet columnar storage format specifically tailored for geospatial data interchange.[68] Unlike PBF, which fundamentally requires reading the entire file to resolve dependencies between ways and relations, GeoParquet organizes data by columns and implements advanced spatial partitioning strategies.[67] This cloud-native format allows game developers and data engineers to perform highly targeted queries—such as extracting only the building footprints within a specific localized bounding box—without downloading or parsing the entire planetary dataset.[69]

The schema provided by Overture also inherently flattens the chaotic tagging structure of OSM. By standardizing entities into explicit themes (such as buildings, transportation networks, administrative divisions, and places of interest) and attaching stable IDs known as the Global Entity Reference System (GERS), GeoParquet dramatically simplifies the data ingestion phase for procedural world generation.[66]

Feature Comparison	OpenStreetMap (XML / PBF)	Overture Maps Foundation (GeoParquet)
Data Structure Architecture	Row-based topological nodes, ways, and relations requiring runtime resolution.	Columnar, pre-processed geometric features natively encoded.
Schema and Tagging	Free-form,	Standardized thematic

	community-driven key/value pairs with massive variance.	modules (Buildings, Transport, Places).
Spatial Indexing & Fetching	Requires secondary processing or PostGIS databases to index.	Built-in cloud-native spatial partitioning and bounding boxes.
Streaming Viability	Low (requires full file dependency parsing in memory).	High (supports byte-range HTTP requests and STAC catalogs).
License Compatibility	ODbL (Open Database License), requiring modified data to remain open.	CDLA-Permissive / ODbL hybrid, allowing private modifications for commercial AI and game systems.[72]

Utilizing specialized SQL-like query engines, developers can dynamically load spatial subsets of the Earth directly into a game engine's memory over HTTP, paving the way for continuously streaming, planetary-scale environments. For example, using DuckDB, a developer can load an HTTP file system and execute a SQL COPY command to extract specific places of interest (like a convenience store) by filtering categories.primary, applying an ILIKE condition to the names.primary column, and clamping the query strictly within defined longitude and latitude boundaries (bbox.xmin and bbox.ymin).[71] This extracts the exact polygonal data needed and outputs it to a standardized format like GeoJSON instantaneously.[71] Similarly, frameworks like Apache Sedona enable reading and writing of GeoParquet files while supporting spatial filter pushdowns, optimizing query performance with profound speedups, bridging the gap between raw data lakes and game engine assets.[70]

Analytical Case Studies in Production

The theoretical constraints of OSM-based world generation are best analyzed through the lens of deployed interactive software, which highlight both the immense potential and the practical limitations of real-world data integration. Over the years, the gaming industry has seen numerous implementations of OSM data, ranging from fitness trackers like *CityStrides* and *Wandrer* (which depend on accurate highway classification to track player movement), to location-based augmented reality games such as *Minecraft Earth* and the original framework for *Monopoly City Streets*.[73] Examining contemporary deep integrations provides critical insights into the architecture of modern GIS game development.

Case Study: Infection Free Zone

Infection Free Zone represents a significant commercial deployment of OSM data within the

real-time strategy and base-building genre.[74] The application allows users to query any geographic location via a text search, triggering a live download of OSM data which is instantaneously converted into a playable 3D environment where existing real-world buildings are re-contextualized as resources, shelters, or defensive hardpoints against procedural zombie hordes.[75]

The architectural analysis of the software reveals both triumphs and specific hurdles. The developers effectively utilize population density maps—such as WorldPop raster data—to drive zombie cluster spawns, ensuring dense urban areas are appropriately dangerous while rural areas remain sparse.[13] Furthermore, by resolving OSM tags like building:material (differentiating brick, glass, and metal) into physical attributes and PBR materials, the game dictates whether a structure offers high or low defensive value, seamlessly translating GIS semantics into core gameplay mechanics.[12]

However, the software explicitly exposes the limitations of isolated vector data. Because it relies almost entirely on the topological footprint without deeply integrated, high-resolution DEM heightmaps, the resulting playable areas are conspicuously flat.[43] Terrains that are fundamentally defined by verticality in reality (such as hills or coastal beaches) are reduced to planar grids, and granular environmental details not typically tagged in OSM (like specific beach sand or micro-topography) are absent, generating a visually homogenized aesthetic regardless of the global location chosen.[43] Furthermore, the game highlights the vulnerability of utilizing crowdsourced data; areas lacking dedicated OSM contributors yield barren virtual landscapes, or incorrectly tagged facilities force illogical gameplay outcomes—such as a small kindergarten functioning as a heavily fortified police armory due to a singular erroneous tag entry by a community mapper.[14]

Case Study: A/B Street

Contrasting with traditional entertainment, *A/B Street* operates as an open-source traffic simulation and urban planning platform built meticulously upon OSM data.[39] Its architecture underscores the extreme geometric complexity required to parse road networks accurately for systemic simulation.[38]

Standard pathfinding algorithms, such as classic Dijkstra's algorithm, fail dramatically in real-world simulations due to complex multi-road turn restrictions encoded in OSM relations.[38] *A/B Street* resolves this by synthesizing contraction hierarchies and executing heavily customized heuristics to interpret standard lanes and turn:lanes tags accurately.[38] The engine physically splits contiguous road surfaces into distinctly rendered lane geometries—accommodating dedicated bicycle paths, street parking lanes, and pedestrian sidewalks with high fidelity.[38]

Achieving this requires a non-trivial map model platform capable of detecting intersection bounds automatically, clipping the procedural geometry, and deducing turning trajectories based entirely on an aggregation of adjacent node metadata.[38] The resulting open-source code represents one of the most advanced, engine-agnostic GIS parsing systems available, proving that deeply semantic, rule-bound world generation is possible given sufficient

algorithmic rigor. The software further demonstrates how developers can modularize GIS processing; by extracting the core geometry utility libraries, the traffic demand generators, and the widgetry UI library into optional files, *A/B Street* allows clients to opt into different data layers, acting as a foundational platform for future hackathons and custom engine importers.[38]

Conclusion

The integration of OpenStreetMap data into game engines represents a monumental leap in procedural world generation, moving the industry from handcrafted artistic illusion to physically anchored simulation. However, achieving an accurate game world based on real-world geography is not a matter of simple data importation. It demands a highly sophisticated architectural pipeline operating across multiple domains of computer science. The raw topological structures of nodes, ways, and relations must be intercepted, subjected to rigorous mathematical projections to resolve Earth's geodetic curvature, and processed through complex tessellation algorithms to convert non-convex boundaries and multipolygons into renderable geometry.

Techniques such as the Straight Skeleton algorithm for procedural roof generation and dynamic spline weaving for complex road intersections act as the critical computational bridges between abstract GIS data and tangible visual fidelity. While middleware tools like Cesium for Unreal and UtyMap for Unity have dramatically lowered the barrier to entry, deep optimization strategies regarding memory streaming, garbage collection, and 64-bit floating-point precision remain absolutely paramount for ensuring stability at a planetary scale. As the landscape evolves, the friction of parsing raw OSM XML and PBF streams is steadily yielding to highly optimized, cloud-native formats like GeoParquet, championed by the Overture Maps Foundation. By combining the community-driven breadth of OpenStreetMap with the rigorous, columnar efficiency of modern data lakes, the technical constraints governing geospatial rendering will continue to diminish. The ultimate realization of the digital twin within interactive entertainment relies entirely on mastering these computational geometry pipelines, fusing deep semantic mapping metadata seamlessly with the high-performance rendering capabilities of modern game engines.

Works cited

1. Map features - OpenStreetMap Wiki, accessed May 22, 2026, https://wiki.openstreetmap.org/wiki/Map_features
2. OSM2World, accessed May 22, 2026, https://fosdem.org/2026/events/attachments/BMMGNT-osm2world_3d_rendering_openstreetmap_data/slides/267176/osm2world_49jn01v.pdf
3. unkei/myosm: Map study - parse and render OpenStreetMap · GitHub, accessed May 22, 2026, https://github.com/unkei/myosm
4. Behavioural Effects of Spatially Structured Scoring Systems in Location-Based Serious Games—A Case Study in the Context of OpenStreetMap - MDPI, accessed May 22, 2026, https://www.mdpi.com/2220-9964/9/2/129
5. Elements - OpenStreetMap Wiki, accessed May 22, 2026, https://wiki.openstreetmap.org/wiki/Elements

6. Node - OpenStreetMap Wiki, accessed May 22, 2026, https://wiki.openstreetmap.org/wiki/Node
7. Georeferencing a Level in Unreal Engine - Epic Games Developers, accessed May 22, 2026, https://dev.epicgames.com/documentation/unreal-engine/georeferencing-a-level-in-unreal-engine
8. How To Build Roads Procedurally In Unity with the Splines Package - YouTube, accessed May 22, 2026, https://www.youtube.com/watch?v=ZiHH_BvjoGk
9. How to Extrude a Custom Mesh along Splines in Unity | by Lukas Kuppers - Medium, accessed May 22, 2026, https://medium.com/@lkuppers11/how-to-extrude-a-custom-mesh-along-splines-in-unity-833a97440c1b
10. Relation - OpenStreetMap Wiki, accessed May 22, 2026, https://wiki.openstreetmap.org/wiki/Relation
11. Tags - OpenStreetMap Wiki, accessed May 22, 2026, https://wiki.openstreetmap.org/wiki/Tags
12. Map data question :: Infection Free Zone General Discussions - Steam Community, accessed May 22, 2026, https://steamcommunity.com/app/1465460/discussions/0/834997729420711200/
13. A post-apocalyptic survival game built on real-world geography, made with AI - Reddit, accessed May 22, 2026, https://www.reddit.com/r/aigamedev/comments/1rl22g1/a_postapocalyptic_survival_game_built_on/
14. Infection Free Zone Review: Defending My Hometown Against Zombies and Bad AI, accessed May 22, 2026, https://www.neonlightsmedia.com/blog/infection-free-zone-early-access-review-2026
15. OsmSharp, accessed May 22, 2026, http://www.osmsharp.com/
16. Software libraries - OpenStreetMap Wiki, accessed May 22, 2026, https://wiki.openstreetmap.org/wiki/Software_libraries
17. Render a building in 3D from OpenStreetMap data - Jacopo Farina's blog, accessed May 22, 2026, https://jacopofarina.eu/posts/render-building-from-openstreetmap/
18. A Guide to Coordinate Reference Systems for Game Developers - Esri, accessed May 22, 2026, https://www.esri.com/arcgis-blog/products/maps-sdk/developers/a-guide-to-coordinate-reference-systems-for-game-developers
19. Terrain Processor: Tutorial - Bohemia Interactive Community Wiki, accessed May 22, 2026, https://community.bistudio.com/wiki/Terrain_Processor:_Tutorial
20. Spatial references | ArcGIS Maps SDK for Unreal Engine - Esri Developer, accessed May 22, 2026, https://developers.arcgis.com/unreal-engine/spatial-reference-and-data-analysis/spatial-references/
21. Converting UTM to Lat Long inside a Game Engine(Untiy 3d) - GIS StackExchange, accessed May 22, 2026,

https://gis.stackexchange.com/questions/170530/converting-utm-to-lat-long-inside-a-game-engineuntiy-3d
22. LatLong Projection - Shader Graph Basics - Episode 32 - YouTube, accessed May 22, 2026, https://www.youtube.com/watch?v=cjTjj-dVGCE
23. The best way to have the entire world in UE5? - Epic Developer Community Forums, accessed May 22, 2026, https://forums.unrealengine.com/t/the-best-way-to-have-the-entire-world-in-ue5/684737
24. Procedurally Generated Road Networks and Intersections - Unity Discussions, accessed May 22, 2026, https://discussions.unity.com/t/procedurally-generated-road-networks-and-intersections/532867
25. Automatic 3D Building Model Generation from Airborne LiDAR Data and OpenStreetMap Using Procedural Modeling - MDPI, accessed May 22, 2026, https://www.mdpi.com/2078-2489/14/7/394
26. Generate 3D City Models from OpenStreetMap (OSM) with Python - YouTube, accessed May 22, 2026, https://www.youtube.com/watch?v=mr8BpLhu6EU
27. Briganti-Games/Straight-Skeleton-Generator: An implementation of the Straight Skeleton algorithm for Unity in C#. · GitHub, accessed May 22, 2026, https://github.com/Briganti-Games/Straight-Skeleton-Generator
28. Exploring procedural generation of buildings - DiVA portal, accessed May 22, 2026, https://www.diva-portal.org/smash/get/diva2:1480518/FULLTEXT01.pdf
29. Syomus/ProceduralToolkit: Procedural generation library for Unity - GitHub, accessed May 22, 2026, https://github.com/Syomus/ProceduralToolkit
30. Procedurally creating a roof? : r/gamedev - Reddit, accessed May 22, 2026, https://www.reddit.com/r/gamedev/comments/u24zgi/procedurally_creating_a_roof/
31. How to calculate uvs for roof procedurally generated? - Unity Discussions, accessed May 22, 2026, https://discussions.unity.com/t/how-to-calculate-uvs-for-roof-procedurally-generated/106590
32. A Procedural Generation Method of Urban Roads Based on OSM - ResearchGate, accessed May 22, 2026, https://www.researchgate.net/publication/363625340_A_Procedural_Generation_Method_of_Urban_Roads_Based_on_OSM
33. Example-Driven Procedural Urban Roads - Computer Science Purdue, accessed May 22, 2026, https://www.cs.purdue.edu/cgvlab/papers/aliaga/cgf15.pdf
34. Best way to make spline roads? - Blueprint - Epic Developer Community Forums, accessed May 22, 2026, https://forums.unrealengine.com/t/best-way-to-make-spline-roads/459290
35. Finding Junctions in Spline-based Road Generation - Diva-Portal.org, accessed May 22, 2026, https://www.diva-portal.org/smash/get/diva2:1675311/FULLTEXT02
36. Roads generated in 3D from OpenStreetMap data - they are drivable! - Reddit, accessed May 22, 2026, https://www.reddit.com/r/proceduralgeneration/comments/1ku48f2/roads_genera

ted_in_3d_from_openstreetmap_data/
37. Create a fully procedural street/path generation algorithm : r/unrealengine - Reddit, accessed May 22, 2026, https://www.reddit.com/r/unrealengine/comments/1juyg5q/create_a_fully_procedural_streetpath_generation/
38. Exporting - A/B Street, accessed May 22, 2026, https://a-b-street.github.io/docs/tech/map/platform.html
39. A/B Street, an OSM-based map viewer and traffic simulator : r/openstreetmap - Reddit, accessed May 22, 2026, https://www.reddit.com/r/openstreetmap/comments/cdjcm9/ab_street_an_osmbased_map_viewer_and_traffic/
40. Related work · a-b-street osm2streets · Discussion #195 - GitHub, accessed May 22, 2026, https://github.com/a-b-street/osm2streets/discussions/195
41. Is there a way to get (terrain) heightmap with arbitrary size of area from osm? - Reddit, accessed May 22, 2026, https://www.reddit.com/r/openstreetmap/comments/cvyw50/is_there_a_way_to_get_terrain_heightmap_with/
42. Hey all. noobie here. Wanting to create heighmaps from OSM data, accessed May 22, 2026, https://community.openstreetmap.org/t/hey-all-noobie-here-wanting-to-create-heighmaps-from-osm-data/88348
43. Kinda disappointed (infection free zone) : r/BaseBuildingGames - Reddit, accessed May 22, 2026, https://www.reddit.com/r/BaseBuildingGames/comments/1c3l896/kinda_disappointed_infection_free_zone/
44. Collect and combine open elevation data to use for contour lines / 3D terrain model / hillshading etc. · Issue #4163 - GitHub, accessed May 22, 2026, https://github.com/organicmaps/organicmaps/issues/4163
45. How can I export elevation layer data and OpenStreetMap layer data as shapefile in QGIS?, accessed May 22, 2026, https://gis.stackexchange.com/questions/363694/how-can-i-export-elevation-layer-data-and-openstreetmap-layer-data-as-shapefile
46. [RELEASED] CityGen3D | Procedural city generation from map data - Unity Discussions, accessed May 22, 2026, https://discussions.unity.com/t/released-citygen3d-procedural-city-generation-from-map-data/689946?page=30
47. Unreal Engine - OpenStreetMap Wiki, accessed May 22, 2026, https://wiki.openstreetmap.org/wiki/Unreal_Engine
48. Cesium for Unreal v1 Quickstart, accessed May 22, 2026, https://cesium.com/learn/unreal/unreal-quickstart-v1-x/
49. Cesium for Unreal Quickstart, accessed May 22, 2026, https://cesium.com/learn/unreal/unreal-quickstart/
50. cesium-unreal-samples/README.md at main - GitHub, accessed May 22, 2026, https://github.com/CesiumGS/cesium-unreal-samples/blob/main/README.md
51. Cesium for Unreal Tutorial 01 - Quickstart - YouTube, accessed May 22, 2026,

https://www.youtube.com/watch?v=BX37A949tiw
52. What's New in Cesium for Unreal - YouTube, accessed May 22, 2026, https://www.youtube.com/watch?v=Y5kCQt8lCBw
53. Unreal Engine 5.7 and Cesium "Maximum Cached Bytes", accessed May 22, 2026, https://community.cesium.com/t/unreal-engine-5-7-and-cesium-maximum-cached-bytes/45557
54. OpenStreetMap Plugin For Unreal Engine Released - GameFromScratch.com, accessed May 22, 2026, https://gamefromscratch.com/openstreetmap-plugin-for-unreal-engine-released/
55. GitHub - ue4plugins/StreetMap: Import OpenStreetMap data into Unreal Engine, accessed May 22, 2026, https://github.com/ue4plugins/StreetMap
56. GitHub - reinterpretcat/utymap: Highly customizable library for procedural world generation based on real map data, accessed May 22, 2026, https://github.com/reinterpretcat/utymap
57. Dynamic 3D world rendering using Unity3D - OpenStreetMap Community Forum, accessed May 22, 2026, https://community.openstreetmap.org/t/dynamic-3d-world-rendering-using-unity3d/70168
58. Unity - OpenStreetMap Wiki, accessed May 22, 2026, https://wiki.openstreetmap.org/wiki/Unity
59. Procedurally Generate Entire Buildings With Unity Splines - YouTube, accessed May 22, 2026, https://www.youtube.com/watch?v=FYmudT12NcI
60. Paul Schrum: Godot GIS - We Ain't Playin' #GodotCon2023 - YouTube, accessed May 22, 2026, https://www.youtube.com/watch?v=W3bR4TdDr8o
61. RodZill4/godot-openstreetmap: Rendering 3d scenes from openstreetmap data... - GitHub, accessed May 22, 2026, https://github.com/RodZill4/godot-openstreetmap
62. Map Viewer - Openstreetmap or other tilemaps - Plugins - Godot Forum, accessed May 22, 2026, https://forum.godotengine.org/t/map-viewer-openstreetmap-or-other-tilemaps/62177
63. Open Street Maps Plugin I Made for Godot (It will be released as Open-Source soon) - Reddit, accessed May 22, 2026, https://www.reddit.com/r/godot/comments/ze874w/open_street_maps_plugin_i_made_for_godot_it_will/
64. OSM workflow for small cities with incomplete data : r/godot - Reddit, accessed May 22, 2026, https://www.reddit.com/r/godot/comments/1pvqlqb/osm_workflow_for_small_cities_with_incomplete_data/
65. What's the difference between this and Open Street Maps? | Hacker News, accessed May 22, 2026, https://news.ycombinator.com/item?id=36880418
66. FAQs - Overture Maps Foundation, accessed May 22, 2026, https://overturemaps.org/about/faq/
67. Go Cloud Native! Overture GeoParquet, From Object Store To Feature Layer Via

Online Notebook - Esri Community, accessed May 22, 2026, https://community.esri.com/t5/etl-patterns-blog/go-cloud-native-overture-geoparquet-from-object/ba-p/1371965
68. GeoParquet, accessed May 22, 2026, https://geoparquet.org/
69. Why We Chose GeoParquet: Breaking Down Data Silos at Overture Maps, accessed May 22, 2026, https://overturemaps.org/blog/2025/why-we-chose-geoparquet-breaking-down-data-silos-at-overture-maps/
70. Harnessing Overture Maps Data: Apache Sedona's Journey from Parquet to GeoParquet, accessed May 22, 2026, https://wherobots.com/blog/wherobots-provides-ready-to-use-spatially-indexed-overture-maps/
71. Quickstart - Overture Maps Documentation, accessed May 22, 2026, https://docs.overturemaps.org/getting-data/
72. OpenStreetMap vs. Overture Maps: Which Is Right for Your Project? - GISCARTA, accessed May 22, 2026, https://giscarta.com/blog/openstreetmap-vs-overture-maps-which-is-right-for-your-project
73. Games - OpenStreetMap Wiki, accessed May 22, 2026, https://wiki.openstreetmap.org/wiki/Games
74. Game with a Open Street Map Real World MAPS??? | Infection Free Zone - YouTube, accessed May 22, 2026, https://www.youtube.com/watch?v=pNgRYX06sPs
75. Buy Infection Free Zone | Xbox, accessed May 22, 2026, https://www.xbox.com/en-US/games/store/infection-free-zone/9pls38pr0fkj
76. Our game, Infection Free Zone, has just launched on Kickstarter! : r/BaseBuildingGames, accessed May 22, 2026, https://www.reddit.com/r/BaseBuildingGames/comments/pufr2y/our_game_infection_free_zone_has_just_launched_on/
77. How To Use The Command Console - Infection Free Zone - YouTube, accessed May 22, 2026, https://www.youtube.com/watch?v=5LOhb1N52SU
78. Infection Free Zone - Official Gameplay Overview - YouTube, accessed May 22, 2026, https://www.youtube.com/watch?v=8KP5xxwRFTM
79. I Hate this!!! :: Infection Free Zone General Discussions - Steam Community, accessed May 22, 2026, https://steamcommunity.com/app/1465460/discussions/0/6026443283692900735/
80. Game "Infection Free Zone" maps based on OSM - OpenStreetMap Community Forum, accessed May 22, 2026, https://community.openstreetmap.org/t/game-infection-free-zone-maps-based-on-osm/8877
81. A/B Street, accessed May 22, 2026, https://a-b-street.github.io/docs/software/abstreet.html